

ESP32S3 GUI-Controlled Power Supply

Using LVGL and FreeRTOS

Filip Radojević

May 2025

Contents

1	Overview	2
2	Schematic of the system	2
3	System Initialization	3
4	FreeRTOS Task Architecture	3
4.1	Encoder Task	3
4.2	CAN Receive Task	4
4.3	Power Supply Task	4
4.4	CAN Watchdog Task	5
4.5	LVGL Display Task	5
5	Display Initialization — LVGL_SCREEN.c	6

This project demonstrates an embedded system implemented on the ESP32-S3 microcontroller. The system integrates hardware peripherals, a graphical display powered by the LVGL library, and a real-time control loop using FreeRTOS. Its main purpose is to interact with a power supply device via the CAN bus and display/control voltage and current parameters through a user-friendly GUI on an ILI9341 TFT 240x320 display.

Task flow diagram for the i9341 module. The diagram shows a hierarchy of tasks: Task_lvgl_il9341 (orange dot) at the top, which receives input from Task_watchdog (red dot), Task_encoder (yellow dot), and Task_pwr_supply (green dot). Task_watchdog has a 250ms self-loop. Task_encoder receives input from three sources: Encoder_SW_ISR, Encoder_SW_ISR, and Encoder_IMPULSE_ISR. Task_pwr_supply receives input from CAN_Receive_ISR. Both Task_encoder and Task_pwr_supply have associated queues (xQueueSetEncoder and xQueueSetLvgl) and semaphores (QueueSetEncoder and xQueueSetLvgl). A legend on the right lists the tasks and their priorities: MIN_Prio_Task (green), MEDIUM_LOW_Prio_Task (yellow), MEDIUM_Prio_Task (yellow), MEDIUM_HIGH_Prio_Task (orange), and MAX_Prio_Task (red).

```

graph TD
    Task_lvgl_il9341[Task_lvgl_il9341]
    Task_watchdog[Task_watchdog]
    Task_encoder[Task_encoder]
    Task_pwr_supply[Task_pwr_supply]
    Task_can_receive[Task_can_receive]

    Task_watchdog -- 250ms --> Task_watchdog
    Task_watchdog --> Task_lvgl_il9341
    Task_encoder --> Task_lvgl_il9341
    Task_pwr_supply --> Task_lvgl_il9341
    Task_encoder --> Task_can_receive

    Encoder_SW_ISR1[Encoder_SW_ISR] --> Task_encoder
    Encoder_SW_ISR2[Encoder_SW_ISR] --> Task_encoder
    Encoder_IMPULSE_ISR[Encoder_IMPULSE_ISR] --> Task_encoder
    CAN_Receive_ISR[CAN_Receive_ISR] --> Task_can_receive

    subgraph xQueueSetLvgl [xQueueSetLvgl]
        lvgl_voltage_queue
        lvgl_current_queue
        lvgl_update_queue
        watchdog_semaphore
        lvgl_bolding_update
    end

    subgraph QueueSetEncoder [QueueSetEncoder]
        encoder_semaphore
        switch_semaphore
        command_semaphore
    end

    subgraph xQueueSetEncoder [xQueueSetEncoder]
        encoder_semaphore
        switch_semaphore
        command_semaphore
    end
  
```

Legend:

- MIN_Prio_Task (green)
- MEDIUM_LOW_Prio_Task (yellow)
- MEDIUM_Prio_Task (yellow)
- MEDIUM_HIGH_Prio_Task (orange)
- MAX_Prio_Task (red)

2

3 System Initialization

Upon startup, all peripherals and software components are initialized in the `main()` function. The following modules are configured:

- **Hardware Peripherals:** LED, CAN (TWAI) interface, timer, rotary encoder, and ILI9341 display.
- **Graphics Library (LVGL):** Initialized with input and display drivers.
- **RTOS Synchronization Objects:** Semaphores and multiple queues including queue sets for task synchronization and communication.

Three semaphores are used to handle different system events:

```
SemaphoreHandle_t encoder_semaphore
SemaphoreHandle_t switch_semaphore
SemaphoreHandle_t command_semaphore
```

These semaphores are triggered inside interrupt service routines (ISRs) associated with the encoder signals, encoder button, and periodic timer interrupts.

4 FreeRTOS Task Architecture

The system functionality is divided into the following tasks:

4.1 Encoder Task

The `task_encoder` is responsible for handling input from a rotary encoder and its integrated push-button. It monitors a `QueueSet` consisting of semaphores that signal different types of events: encoder rotation, button presses, and periodic command triggers.

Upon receiving a signal from any of these semaphores, the task performs one of the following actions:

- **Encoder Rotation (`encoder_semaphore`):**
 - Reads the CLK and DT pins to determine the direction of encoder rotation.
 - Implements software debounce by ignoring pulses occurring within a `STEP_DEBOUNCE_INTERVAL` (17.5 ms) window.
 - If the encoder button is not pressed, changes the current selection (voltage or current adjustment mode).
 - If the encoder button is pressed, modifies the value of the currently selected parameter:
 - * For voltage: updates the internal voltage step counter, calculates the new voltage, clamps it within the predefined range, and sends the updated value via CAN.
 - * For current: similarly updates the current limit using the encoder steps and sends the result over CAN.

- In both cases, the updated value is pushed to the corresponding LVGL queue to refresh the UI.
- **Encoder Button Press (`switch_semaphore`):**
 - Reads the encoder push-button pin state.
 - Performs software debouncing using a short interval defined by `DEBOUNCE_DELAY_US`.
 - If a valid press is detected, toggles between voltage and current adjustment modes.
 - Sends the new mode to the UI using the `lvgl_button_pressed` queue.
- **Periodic Command Trigger (`command_semaphore`):**
 - Periodically re-sends the currently selected voltage value over CAN.
 - This mechanism ensures that the power supply does not reset due to inactivity, by maintaining communication.

4.2 CAN Receive Task

The `task_can_receive` waits for incoming CAN messages. These messages are pushed into the `xQueueCan` queue from the CAN ISR. The task then forwards valid data to the power supply task.

4.3 Power Supply Task

The `task_pwr_supply` is responsible for requesting and processing power-related parameters from the external power supply unit via the CAN bus. The task operates in a continuous loop and follows these key steps:

1. It begins by constructing and sending a CAN message to request real-time data from the power supply.
2. After transmitting the request, the task blocks on the `xQueueCan` queue and waits for a response with a predefined timeout.
3. Upon receiving a message, it checks whether the message contains either live measurement data (`CAN_REQUEST_VALUES_ID`) or indicates the end of the current data frame (`CAN_END_OF_REQUEST_VALUES_ID`).
4. If the response is valid, the message payload is parsed via the `parse_statistics` function to extract parameters such as output voltage, current, temperature, and efficiency.
5. Once the complete data frame is received, and enough time has elapsed since the last update (to reduce GUI refresh overhead), a structured `lvgl_data_t` object is populated with the latest values.
6. This object is then sent to the GUI update queue (`lvgl_update_queue`), allowing the LVGL task to refresh the on-screen display with updated information.

This task ensures the GUI is periodically updated with meaningful and fresh telemetry from the power supply, while minimizing unnecessary traffic and CPU usage through timed updates and data validation logic.

4.4 CAN Watchdog Task

The `task_can_watchdog` is a supervisory task that monitors the health of the CAN communication link. It operates on a periodic timer and relies on a shared flag variable (`can_alive_flag`) which is updated by the `task_can_receive` task whenever a new CAN message is successfully received.

The key functionality of this task is as follows:

- Every 250 milliseconds, the task wakes up from a delay and checks the state of the `can_alive_flag`.
- If the flag has not been set in the current cycle, it assumes that no valid CAN message has been received within the expected timeframe.
- In that case, it signals a failure by giving the `watchdog_semaphore`, which may be handled by another task responsible for error recovery or system reset logic.
- Regardless of the result, the flag is reset to `false` at the end of each cycle to prepare for the next monitoring interval.

This mechanism provides a simple yet effective heartbeat-style monitoring of the CAN interface, allowing the system to detect bus faults, disconnections, or peripheral failures and respond appropriately.

4.5 LVGL Display Task

The `task_lvgl_ili9341` task is responsible for handling the user interface using the LVGL graphics library in combination with the ILI9341 display driver. It utilizes a `QueueSet` mechanism to manage multiple sources of GUI updates in a single task context, ensuring synchronization and responsiveness.

This task listens on a set of queues and semaphores which reflect various types of input or system status updates. Each input source triggers a different behavior:

- **Voltage Queue** (`lvgl_voltage_queue`):
 - Receives a voltage value.
 - Converts it to a human-readable value and updates the voltage label and arc on the screen.
- **Current Queue** (`lvgl_current_queue`):
 - Receives a current limit value.
 - Converts and displays it similarly to voltage.
- **Display Update Queue** (`lvgl_update_queue`):
 - Receives a complete set of system statistics via a `lvgl_data_t` structure.

- Calls the `update_lvgl_display` function, which draws voltage, current, power, temperature, and efficiency.
- **Bold Label Selection Queue** (`lvgl_bolding_update`):
 - Handles UI highlighting to indicate the selected parameter (voltage or current).
 - Updates the appearance of labels accordingly.
- **Button Pressed Queue** (`lvgl_button_pressed`):
 - Handles rotary encoder or button press events.
 - Updates the screen to reflect parameter changes initiated by the user.
- **Watchdog Semaphore** (`watchdog_semaphore`):
 - Triggered when the CAN watchdog task detects a communication failure.
 - Activates a visual error indication (e.g., blinking LED or error message on display).

All GUI-related updates are protected by a mutex to ensure thread-safe access to LVGL's internal state. The `lv_timer_handler()` function is invoked after each update to refresh the display.

5 Display Initialization — `LVGL_SCREEN.S.C`

The graphical interface is initialized and styled in a custom screen layout file. This file is derived from the automatically generated `screen.c` provided by the EEZ Studio tool, which exports graphical LVGL layouts. The code structure remains largely the same, with only a few additions to support runtime display orientation and object references.

During boot, the screen is cleared, rotated, and stylized:

```
// Clear the current screen
lv_obj_clean(lv_scr_act());

// Save global display pointer
global_disp = disp;

// Set display to landscape
lv_disp_set_rotation(disp, LV_DISP_ROT_90);

// Set up main screen object
lv_obj_t *scr = lv_disp_get_scr_act(disp);

// Create main container
lv_obj_t *obj = lv_obj_create(scr);
objects.main = obj;
```

The only modifications to the original EEZ Studio output include:

- Setting the display rotation to landscape mode using `LV_DISP_ROT_90`.
- Saving a reference to the active display in `global_disp` for later use.

- Creating and storing a reference to the main container object for dynamic updates at runtime.

This ensures correct screen orientation and styling, providing a clear and intuitive UI layout fully compatible with the dynamic behavior defined elsewhere in the firmware.